



SMM

Project (Student 3)

Prepared By:
Suleman (22i-8807)
SE-D

Date: 10th May, 2026

Table of Contents

1. Task 6 — KLM Usability Evaluation.....	2
1.1 Operator constants.....	2
1.2 Scenario coverage.....	2
1.3 Scenario 1 — Sign in.....	3
1.4 Scenario 2 — Register a Student Account.....	4
1.5 Scenario 3 — Administrator adds a new Society.....	5
1.6 Scenario 4 — Society Head creates an Event.....	6
1.7 Scenario 5 — Student registers for an Event.....	7
1.8 Summary of all scenarios.....	8
1.9 Findings and recommendations.....	8
2. Task 7 — COCOMO Cost Estimation.....	9
2.1 Size measurement.....	9
2.2 Mode justification.....	10
2.3 Basic COCOMO.....	10
2.4 Intermediate COCOMO.....	11
2.5 Detailed COCOMO.....	13
2.6 Recommendation.....	14
2.7 Reality check against actual effort.....	15
3. Task 8 — Documentation Ratio.....	15
3.1 Methodology.....	15
3.2 Counts.....	15
3.3 Documentation ratio.....	16
3.4 Interpretation.....	16
Appendix A — Per-file Lines-of-Code table.....	17

1. Task 6 — KLM Usability Evaluation

1.1 Operator constants

The Keystroke-Level Model (Card, Moran and Newell, 1980) was applied using the four operators specified in the task brief.

Operator	Meaning	Time (ms)
K	Keystroke or mouse-button press	280
M	Mental preparation	1350
P	Pointing (mouse movement)	1100
H	Hand movement, keyboard to mouse or vice-versa	400

Convention used in this report. As in the original CMN formulation, a mouse click is considered as the sequence P + K (movement + button press). Because the brief does not include a separate “B” operator, K is used for both keyboard keystrokes and mouse-button presses. M is inserted before each cognitive sub-task and before any string that the user needs to remember (for example, an email address or an event title). Heuristic pruning rules from Card et al. (1980) were applied to eliminate M operators predicted by an obvious primary action.

1.2 Scenario coverage

KLM is task-based (and not screen-based) and so the evaluation is based on one representative scenario per role, plus the two shared authentication forms. All five scenarios combined exercise every form file in the application.

No.	Scenario	Form(s) reached
1	Sign in to the application	LoginForm
2	Register a new student account	RegisterForm

No.	Scenario	Form(s) reached
3	Administrator adds a new society	AdminDashboard, Societies tab, Create Society sub-tab
4	Society Head creates a new event	SocietyDashboard, Events page, EventFormDialog
5	Student registers for an upcoming event	StudentDashboard, Events page, confirmation dialog

For Scenarios 3 to 5 the user is assumed to be already authenticated, so the login time is not counted twice.

1.3 Scenario 1 — Sign in

Inputs: email i228897@nu.edu.pk (19 characters), password Pass1234 (8 characters). **Starting position:** hand on mouse.

Step	Operator	Description	Time (ms)
1	M	Plan to sign in	1350
2	P	Move to Email field	1100
3	K	Click	280
4	H	Mouse to keyboard	400
5	M	Recall email address	1350
6	19 K	Type email	5320
7	K	Press Tab	280
8	M	Recall password	1350
9	8 K	Type password	2240
10	H	Keyboard to mouse	400
11	P	Move to SIGN IN button	1100

Step	Operator	Description	Time (ms)
12	K	Click	280

Operator totals: 3 M, 30 K, 2 P, 2 H. **Total time:** 4050 + 8400 + 2200 + 800 = **15 450 ms (15.45 s).**

1.4 Scenario 2 — Register a Student Account

Inputs (normal lengths) First name (5) Last name (6) Email (15) Registration number (8) Phone (12) Department dropdown (left at default) Semester dropdown (changed to "Semester 3") Password (8) Confirm password (8) Last step is hit CREATE ACCOUNT. Starting position: hand on mouse, focus on form.

Step	Operator	Description	Time (ms)
1	M	Plan registration	1350
2	P, K	Click First Name field	1380
3	H	Mouse to keyboard	400
4	5 K	Type first name	1400
5	K	Tab	280
6	M, 6 K	Recall and type last name	3030
7	K	Tab	280
8	M, 15 K	Recall and type email	5550
9	K	Tab	280
10	M, 8 K	Recall and type registration number	3590
11	K	Tab	280
12	M, 12 K	Recall and type phone	4710

Step	Operator	Description	Time (ms)
13	2 K	Tab past Department, Tab into Semester	560
14	H, P, K	Mouse, click Semester dropdown	1780
15	M, P, K	Locate and click "Semester 3"	2730
16	H	Mouse to keyboard	400
17	K	Tab to Password	280
18	M, 8 K	Recall and type password	3590
19	K	Tab to Confirm	280
20	8 K	Re-type password	2240
21	H, P, K	Mouse, click CREATE ACCOUNT	1780

Operator totals: 7 M, 73 K, 4 P, 4 H. **Total time:** 9450 + 20 440 + 4400 + 1600 = **35 890 ms (35.89 s).**

1.5 Scenario 3 — Administrator adds a new Society

The administrator is on the AdminDashboard home page. A society named "Robotics Club" (13 characters) is created in the default category, with description "All things robotics" (19 characters).

Step	Operator	Description	Time (ms)
1	M	Plan task	1350
2	P, K	Click Societies navigation item	1380

Step	Operator	Description	Time (ms)
3	M, P, K	Locate and click Create Society tab	2730
4	P, K	Click Society Name field	1380
5	H	Mouse to keyboard	400
6	M, 13 K	Recall and type "Robotics Club"	4990
7	H, P, K	Mouse, click Description field	1780
8	H	Mouse to keyboard	400
9	M, 19 K	Recall and type description	6670
10	H, P, K	Mouse, click CREATE AND ACTIVATE SOCIETY	1780

Operator totals: 4 M, 37 K, 5 P, 4 H. **Total time:** 5400 + 10 360 + 5500 + 1600 = **22 860 ms (22.86 s).**

1.6 Scenario 4 — Society Head creates an Event

The user is on the SocietyDashboard home page. The user creates an event titled "AI Workshop" (11 characters) with description "Hands-on intro to ML" (20 characters), the default type "Workshop", venue "CS Lab 1" (8 characters), and adjusts the start date with approximately four keystrokes on the date picker.

Step	Operator	Description	Time (ms)
1	M	Plan task	1350
2	P, K	Click Events navigation	1380
3	M, P, K	Locate and click + New Event	2730

Step	Operator	Description	Time (ms)
4	P, K	Click Title field	1380
5	H	Mouse to keyboard	400
6	M, 11 K	Recall and type title	4430
7	H, P, K	Mouse, click Description	1780
8	H	Mouse to keyboard	400
9	M, 20 K	Recall and type description	6950
10	H, P, K	Mouse, click Venue	1780
11	H	Mouse to keyboard	400
12	8 K	Type "CS Lab 1"	2240
13	H, P, K	Mouse, click Start Date picker	1780
14	H	Mouse to keyboard	400
15	M, 4 K	Decide and adjust date	2470
16	H, P, K	Mouse, click CREATE EVENT	1780

Operator totals: 5 M, 30 K, 7 P, 8 H. **Total time:** 6750 + 8400 + 7700 + 3200 = **26 050 ms (26.05 s).**

1.7 Scenario 5 — Student registers for an Event

The user is on the StudentDashboard home page. The user navigates to Events, locates the desired event card, clicks Register, and dismisses the success message-box.

Step	Operator	Description	Time (ms)
1	M	Plan task	1350

Step	Operator	Description	Time (ms)
2	P, K	Click Events navigation	1380
3	M, P, K	Scan cards and click Register	2730
4	M, P, K	Read confirmation and click OK	2730

Operator totals: 3 M, 3 K, 3 P, 0 H. **Total time:** 4050 + 840 + 3300 + 0 = **8190 ms (8.19 s)**.

1.8 Summary of all scenarios

Scenario	K	M	P	H	Total time
1. Sign in	30	3	2	2	15.45 s
2. Register	73	7	4	4	35.89 s
3. Administrator adds society	37	4	5	4	22.86 s
4. Society Head creates event	30	5	7	8	26.05 s
5. Student registers for event	3	3	3	0	8.19 s
Totals	173	22	21	18	108.44 s (1 min 48 s)

1.9 Findings and recommendations

Registration is the heaviest flow at 35.9 s. Most of the time is unavoidable typing (about 20 s of K). Two practical improvements would each save roughly 1.7 to 1.8 seconds:

1. Make the Department and Semester dropdowns fully keyboard-tabbable with first-letter selection. The ComboBox supports this natively when `AutoCompleteSource = ListItems`. This eliminates the H + P + K cluster on the dropdown.
2. Replace First Name and Last Name with a single Full Name field, removing one M, one Tab and one cognitive recall.

The Administrator Add-Society flow (22.9 s) contains two redundant H switches because the Description field sits below the Name field. Reordering the form so that Name and Description are tab-adjacent saves one P + K + H sequence (about 1.8 s).

The Society Create-Event flow is mouse-heavy (eight H operators). Setting `KeyPreview = true` on the dialog and improving the Tab order (Title to Description to Type to Venue to Date) would let an experienced user complete the form with one hand on the keyboard.

The Student Register-for-Event flow is already near-optimal for a three-click confirmation interaction.

2. Task 7 — COCOMO Cost Estimation

2.1 Size measurement

The codebase size was measured by counting non-blank, non-comment-only lines in all hand-written .cs files. Auto-generated *.Designer.cs files and the bin/ and obj/ build directories were excluded.

Metric	Value
Files counted	46
Total physical lines	5040
Blank lines	675
Comment-only lines	56
Source Lines of Code (SLOC)	4309

The size used for COCOMO is therefore **KLOC = 4.309**, rounded to 4.3 in calculations.

2.2 Mode justification

Boehm's COCOMO defines three development modes. The table below evaluates each one against the project's characteristics.

Mode	Typical traits	Fit for this project
Organic	Small, in-house team; familiar problem domain; relaxed deadlines; CRUD-style applications	Good fit — single-developer student project, well-understood domain (university clubs)
Semi-detached	Mixed team experience; medium size; some unfamiliar elements	Poor fit
Embedded	Tight hardware, timing or regulatory constraints	Poor fit

The project is therefore evaluated in **Organic mode**.

2.3 Basic COCOMO

The Basic COCOMO formulas are:

- Effort in person-months: $E = a \times \text{KLOC}^b$
- Development time in months: $\text{TDEV} = c \times E^d$
- Average staff size: $P = E \div \text{TDEV}$

Applied to KLOC = 4.309:

Mode	a	b	c	d	Effort (PM)	TDEV (months)	Average staff
Organic	2.4	1.05	2.5	0.38	11.13	6.25	1.78
Semi-detached	3.0	1.12	2.5	0.35	15.40	6.51	2.37
Embedded	3.6	1.20	2.5	0.32	20.78	6.60	3.15

The Organic estimate gives approximately **11 person-months over 6.25 months with 1.78 average staff**, which is the value used as the Basic-COCOMO baseline.

2.4 Intermediate COCOMO

Intermediate COCOMO refines Basic COCOMO by introducing an Effort Adjustment Factor (EAF) that is the product of fifteen cost-driver multipliers. The Organic coefficients become $a = 3.2$, $b = 1.05$.

The cost-driver ratings used here are justified in the table below.

Driver	Description	Rating	Multiplier	Justification
RELY	Required software reliability	Low	0.88	Student or teaching artefact; failure has no safety or financial consequence
DATA	Database size	Low	0.94	Small SQL Server schema, well under 10 MB of realistic data
CPLX	Product complexity	Nominal	1.00	Routine CRUD logic plus reporting
TIME	Execution-time constraint	Nominal	1.00	Desktop application, no real-time loop
STOR	Main-storage constraint	Nominal	1.00	Modern PCs; plenty of memory
VIRT	Platform volatility	Low	0.87	Stable .NET 8 and SQL Server target
TURN	Computer turnaround	Low	0.87	Interactive development;

Driver	Description	Rating	Multiplier	Justification
				instant compile and run
ACAP	Analyst capability	Nominal	1.00	—
AEXP	Application experience	Nominal	1.00	—
PCAP	Programmer capability	Nominal	1.00	—
VEXP	Virtual-machine experience	Nominal	1.00	—
LEXP	Programming language experience	Nominal	1.00	—
MODP	Modern programming practices	Nominal	1.00	—
TOOL	Use of software tools	High	0.91	Modern IDE plus AI-assisted coding
SCED	Required development schedule	Nominal	1.00	—

The Effort Adjustment Factor is therefore:

$$EAF = 0.88 \times 0.94 \times 0.87 \times 0.87 \times 0.91 = \mathbf{0.570}$$

Applying the Intermediate Organic formula:

Quantity	Calculation	Value
Effort	$3.2 \times 4.309^{1.05} \times 0.570$	8.45 PM
TDEV	$2.5 \times 8.45^{0.38}$	5.63 months

Quantity	Calculation	Value
Average staff	$8.45 \div 5.63$	1.50

The four "Low" drivers and the "High" TOOL rating drag the EAF well below 1.0, reflecting a small, low-risk project built with strong tooling — exactly the conditions Intermediate COCOMO was designed to capture.

2.5 Detailed COCOMO

Detailed COCOMO is an extension of Intermediate with phase-sensitive multipliers and a three-tiered product hierarchy (system, sub-system, module). There is only one product level with only one 4.3-KLOC application in this codebase, and consequently the practical contribution of Detailed COCOMO is the phase distribution of effort and schedule.

Boehm's standard distribution of phases for a small Organic-mode project is applied to the Intermediate effort of 8.45 person-months and Intermediate schedule of 5.63 months.

Phase	Effort share	Effort (PM)	Schedule share	Schedule (months)
Plans and Requirements	6 % *	0.51	10 % *	0.56
Product Design	16 %	1.35	19 %	1.07
Detailed Design	26 %	2.20	combined below	combined
Code and Unit Test	42 %	3.55	(Programming) 63 %	3.54
Integration and Test	16 %	1.35	18 %	1.01
Total (Product Design through Integration and Test)	100 %	8.45	100 %	5.63

* Plans and Requirements is reported on top of the 100 % base, in line with Boehm's convention.

2.6 Recommendation

Variant	Strength	Weakness for this project	Verdict
Basic	Quick, requires no judgement calls	Ignores the productivity boost from AI-assisted tooling and the low-stakes nature of the work; over-estimates by approximately 30 %	Useful as a sanity check
Intermediate	Captures the cost drivers that matter here (low reliability needs, stable platform, strong tooling) without inventing data the project does not have	Still treats the codebase as one undifferentiated module	Recommended for this project
Detailed	Adds phase-sensitive multipliers and multi-level decomposition	The codebase is a single 4-KLOC module, so multi-level decomposition is overkill; per-phase EAFs require data not collected for a student project	Overkill; only the phase distribution is genuinely useful, and that is included in section 2.5

The recommended estimate is the Intermediate-COCOMO estimate: 8.45 person-months in 5.63 months with 1.50 average staff. The Basic Organic estimate (11.13 person-months over 6.25 months) is reported as a sanity-check upper bound and the phase distribution from Detailed COCOMO is used to distribute that effort across milestones.

2.7 Reality check against actual effort

The Code and Unit Test phase can be greatly shortened with AI-assisted development (vibe coding). The 8.45 PM Intermediate figure is taken to be the traditional human

productivity. The actual effort observed for this single-developer project was a small fraction of the Intermediate estimate, and that gap itself is a useful finding: it quantifies the productivity multiplier of AI-assisted development on a CRUD-style desktop application.

3. Task 8 — Documentation Ratio

3.1 Methodology

- Number of files: all .cs files in the repository that are handwritten.
- Exclude files: *.Designer.cs (WinForms designer auto-generated), bin/, obj/.
- A line is a comment line if the non-whitespace content consists only of comment text (i.e., a single line comment starting with // or any line inside a /*... */ block).
- Lines containing code and a trailing comment are treated as code, not comment lines. There is only one such line in the whole code base.
- Total LOC includes blank lines, according to the formula provided in the task brief.

3.2 Counts

Metric	Value
Files analysed	46
Total LOC (including blanks and comments)	5040
Blank lines	675
Source LOC (code)	4309
Comment lines	57 (56 pure comment lines plus 1 line with an inline comment)

3.3 Documentation ratio

The task brief describes the formula as Total LOC / Comment Lines. In the software-engineering literature the standard definition is the opposite -- comment lines as a fraction of total LOC. Both forms are shown below for completeness.

Definition	Calculation	Result
Conventional: Comment ÷ Total LOC	57 ÷ 5040	1.13 %
Conventional, against SLOC only	57 ÷ 4309	1.32 %
As written in the brief: Total LOC ÷ Comment	5040 ÷ 57	88.4 lines of code per comment line

3.4 Interpretation

The ratio of documentation of about 1.1 % is very low by industry standards. Mature open source C# projects typically are in the 10 to 20 % range, including XML-doc comments. What we see here is a signature signature of vibe coded output. Three reasons:

1. AI-generated code prefers self-documenting style — descriptive names and small focused methods — over explicit comments.
2. The few comments that exist are navigation banners within large dashboard files (e.g. section dividers between page builder methods) not behavioural explanations.
3. There are no XML doc-comments at all on the public API surface (Models, Services and Repositories). There are no triple-slash /// tags which drive IntelliSense and tooling like Sandcastle or DocFX.

Implication for hand off and maintenance If this codebase were handed over to another developer, the missing “why” comments — rationale, invariants, edge-case explanations — would dominate the onboarding cost. The cheapest remediation is to add XML doc-comments on the public methods of the Services/ and Data/Repositories/ folders, about 75 methods in total. This will increase the documentation ratio to over 5 % without changing any runtime behaviour .

Appendix A — Per-file Lines-of-Code table

File	Total	Blank	Code	Comments
Data/Database Connection.cs	46	6	40	0
Data/Repositories/AnnouncementRepository.cs	118	11	107	0
Data/Repositories/EventRegistrationRepository.cs	121	12	109	0
Data/Repositories/EventRepository.cs	150	13	137	0
Data/Repositories/IRepository.cs	10	1	9	0
Data/Repositories/MembershipRepository.cs	227	16	211	0
Data/Repositories/SocietyRepository.cs	138	13	125	0
Data/Repositories/StudentRepository.cs	137	12	125	0
Data/Repositories/TaskRepository.cs	139	12	127	0
Data/Repositories/UserRepository.cs	114	11	103	0

File	Total	Blank	Code	Comments
Helpers/PasswordHelper.cs	12	3	9	0
Helpers/TicketHelper.cs	7	1	6	0
Helpers/ValidationHelper.cs	34	9	25	0
Models/Admin.cs	13	3	9	1
Models/Announcement.cs	15	1	14	0
Models/Events.cs	19	1	18	0
Models/EventRegistration.cs	17	1	16	0
Models/MembershipRequest.cs	15	1	14	0
Models/Society.cs	15	1	14	0
Models/SocietyMember.cs	13	1	12	0
Models/SocietyTask.cs	18	1	17	0
Models/Student.cs	16	3	12	1
Models/User.cs	11	1	10	0
Program.cs	35	6	28	1
Properties/Settings.cs	10	2	8	0

File	Total	Blank	Code	Comments
Services/AnnouncementService.cs	57	13	44	0
Services/AuthService.cs	74	16	58	0
Services/EventService.cs	108	25	83	0
Services/MembershipServices	53	14	39	0
Services/ReportService.cs	160	16	144	0
Services/SocietyService.cs	126	29	95	2
Services/TaskService.cs	62	13	49	0
UI/Controls/ModernButton.cs	97	17	78	2
UI/Controls/ModernTextBox.cs	193	27	160	6
UI/Controls/RoundedPanel.cs	47	5	42	0
UI/Controls/StatCard.cs	64	8	56	0
UI/Forms/Admin/AdminDashboard.cs	471	64	399	8
UI/Forms/Shared/BaseShellForm.cs	241	26	209	6

File	Total	Blank	Code	Comments
UI/Forms/Shared/LoginForm.cs	237	32	202	3
UI/Forms/Shared/RegisterForm.cs	213	28	184	1
UI/Forms/Society/AnnouncementFormDialog.cs	72	14	58	0
UI/Forms/Society/EventFormDialog.cs	141	24	117	0
UI/Forms/Society/SocietyDashboard.cs	491	65	414	12
UI/Forms/Society/TaskFormDialog.cs	88	17	71	0
UI/Forms/Student/StudentDashboard.cs	538	75	453	10
UI/Theme/AppTheme.cs	57	5	48	4
Total (46 files)	5040	675	4309	57

End of report.